

Regime-Aware Policy Transfer for Computationally Efficient Adaptation under Network Data Drift

Anonymous Submission
Paper ID: *under double-blind review*

Abstract—Network telemetry is non-stationary, and predictive models trained on an earlier period degrade when operating conditions change. Conventional adaptation responds by retraining, effectively but at the repeated full cost of rebuilding a model. We ask whether adaptation policies learned for previously observed operating regimes can be reused instead of rebuilt. We present Regime-Aware Policy Transfer (RAPT), which associates a learned ensemble policy with each regime, retrieves it when a matching regime recurs, and refreshes it when monitoring shows the stored policy no longer fits. Evaluated against frozen, event-driven and full-retraining baselines on four network streams under a strictly prequential protocol, RAPT reduced adaptation CPU relative to full retraining by 53–88%, but its predictive performance was not uniformly better: it recorded lower macro-F1 on all four streams, and on one stream blind reuse was actively harmful, a failure a larger novelty-refit buffer repairs. On the 5G Campus ablation stream a cost-aware configuration matched full retraining within measurement noise ($\Delta_{\text{macro-F1}} = -0.0022$, $p = 0.44$) at roughly 23% lower adaptation cost. Two mechanisms were rejected on measured evidence: a relative-degradation trigger that cannot fire on persistently poor policies, and an incremental learner dominated on both accuracy and cost.

Index Terms—concept drift, network telemetry, online learning, adaptive ensembles, regime-aware learning, computationally efficient adaptation

I. INTRODUCTION

Operational network data rarely holds still. Load, radio conditions, subscriber mobility and configuration changes all shift the distribution of the measurements that monitoring systems observe, and a classifier fitted to one period of a 5G testbed or an ISP netflow archive will not stay accurate indefinitely. The standard remedy is to retrain when performance falls. That works, but it is paid for repeatedly, and on high-rate telemetry the cumulative rebuild cost can exceed the cost of serving predictions; lightweight detection-and-adaptation frameworks have been proposed specifically for constrained devices [16].

Many effective detectors exist for *when* to adapt [7]–[10], [12], [14], [15], but none exploits the structure of the change. Network conditions are frequently *recurring* rather than merely novel: a weekday congestion profile, a radio configuration, or an operator-specific QoS regime may appear, disappear, and return. Recurring concepts motivated some of the earliest work in the field [4], [5], yet the common response remains to relearn, discarding a model that is often still the right one.

This paper studies whether that information can be retained and reused. RAPT maintains a repository of adaptation policies indexed by operating regime. At a regime boundary it checks

whether a policy for the incoming regime is already stored; if so, the stored ensemble is retrieved at near-zero adaptation cost, and otherwise a policy is trained and stored. Because a retrieved policy may no longer fit if the regime’s label relationship has changed, RAPT also monitors its streaming accuracy and can refresh it. RAPT changes when training is paid for, not how the ensemble is trained.

Our answer to whether the substitution is safe is conditional. Where regimes recur with stable label semantics, reuse preserves accuracy while removing most of the adaptation cost; where regimes recur but the feature-to-label relationship has shifted, reuse retrieves a stale policy and degrades performance, and the enhancement that repairs it is what makes the approach viable there. The contributions are a regime-conditioned policy-transfer mechanism that stores and retrieves ensemble policies by regime; a monitoring and refresh strategy whose cost-aware configuration matches full-retraining accuracy on the evaluated stream at a fraction of its cost; an evaluation across four network streams under one prequential protocol against four established drift detectors; and a failure analysis of harmful blind reuse, a trigger that cannot fire by construction, and an incremental learner dominated on both axes.

II. RELATED WORK

A. Concept and Data Drift

Distributional change in streaming data is separated into covariate shift, where $P(X)$ changes while $P(Y | X)$ is stable, and concept drift, where $P(Y | X)$ itself changes [1], [2]. Survey work documents a long history of taxonomies and adaptation strategies [3], beginning with the recurring concepts of STAGGER [4] and the robustness–flexibility trade-off studied by Widmer [5]. The two call for different responses: covariate shift can be absorbed by recalibrating a decision boundary, whereas concept drift invalidates the learned mapping. Recurring drift is common in systems whose load follows periodic patterns yet has received less attention than gradual or abrupt drift [6].

B. Drift Detection

Detectors trigger adaptation rather than performing it. ADWIN compares sub-window means over an adaptive window [7]; Page-Hinkley is a sequential cumulative-sum procedure [8]; DDM and EDD monitor the error rate and its variance [9], [10], while RDDM adds reduced false alarms [11] and an ECDD-EWMA control chart tracks an exponentially weighted

moving average of the error [12]. Label-free detectors test the feature stream instead, using Hoeffding bounds [13], an incremental Kolmogorov–Smirnov statistic [14], or the earth-mover distance [15], with lightweight variants for constrained IoT deployments [16]. They differ in sensitivity and firing frequency, which translates into adaptation cost.

C. Adaptive and Online Learning

Online ensembles adapt by updating constituent models as data arrives. Dynamic weighted majority weights learners by recent accuracy and adds or removes them as the stream evolves [17], an idea disagreement-minimizing methods had already applied to drifting concepts [18]; adaptive random forests extend online bagging with drift-aware tree replacement [6], the accuracy-updated ensemble reweights components under drift [19], and such designs have been specialised to social big-data and IoT streams [20], [21]. Single-pass incremental learners such as Hoeffding trees bound memory and update cost per example [22], and the River library implements the detectors and incremental learners used here [23]. Incremental updating changes *how* a model is updated rather than whether an already-adequate model can be retrieved; Section VI-D reports a comparison in which it did not improve the accuracy–cost trade-off here.

D. Transfer under Recurring Conditions

Transfer learning moves representations or parameters from a source task to a related target task [24]. Retaining rather than discarding learned structure has an older lineage in work on the role of forgetting [25] and the cost of reusing compiled knowledge chunks [26], both observing that relearning what is already known is wasteful. RAPT transfers something narrower: a previously fitted policy and its regime identity are stored and retrieved whole, so the decision is not how to adapt but whether adaptation is needed at all.

III. PROBLEM FORMULATION

Let a stream be a chronological sequence $x_1, x_2, \dots$, where $x_t \in \mathbb{R}^d$ is a feature vector and $y_t \in \{1, \dots, C\}$ its label, available only after the prediction for x_t . The stream is partitioned into fixed-size windows $W_1, \dots, W_T$, and window W_k carries a regime identifier $r_k \in \mathcal{R}$. A regime is a semantically defined operating condition rather than a cluster label, such as a network scenario or a time-of-day block. A *policy* π is a fitted model with its decision rule, and the repository is a mapping $\mathcal{P} : \mathcal{R} \rightarrow \pi$ populated as regimes are encountered. When the stream moves from r_{k-1} to r_k , the controller chooses $a_k \in \{\text{REUSE}, \text{TRAIN}\}$: reuse $\mathcal{P}(r_k)$ if it exists, or train from the recent buffer. With $M(\pi, W)$ a performance measure (macro-F1 here) and $c(\pi)$ the adaptation CPU cost of producing π , the problem is to keep $M(\cdot, W_k)$ close to a freshly trained policy’s while minimizing $\sum_k c(a_k)$. Reuse assumes $P(Y | X, r_k)$ is unchanged since the policy was stored; if the regime identifier recurs but the conditional relationship has shifted, the stale policy is applied with no signal that it is stale. Distinguishing recurrence of the regime

from recurrence of the label relationship is the central difficulty.

IV. PROPOSED METHOD: RAPT

A. Architecture and Repository

RAPT sits between the stream and the model and makes one decision per regime boundary. Each window is predicted with the currently active policy and evaluated once its labels are available, and the policy changes only at a regime transition or when monitoring reports degradation — the prequential discipline of Section V, which prevents a window’s labels from influencing its own prediction. A regime is represented by its semantic identifier rather than a learned clustering, which keeps repository keys stable and avoids the failure mode in which clustering splits one operating condition into several keys and defeats reuse; the mechanism therefore depends on a usable regime signal, an assumption we revisit in Section VIII. The repository stores, per regime, a checkpoint holding the fitted ensemble, its soft-voting weights, and creation and last-access timestamps. Checkpoints are immutable, so reuse retrieves the ensemble by reference rather than copying it, and only the small weight vector is adjusted when calibration applies; retrieval cost is therefore independent of model size, making reuse cheap relative to retraining.

B. Matching and Transfer

Matching is by regime identifier: on a transition to regime r the controller tests whether $r \in \mathcal{P}$. In the enhanced variant a fingerprint-similarity gate also requires the current window’s feature summary to be close to the fingerprint recorded at checkpoint creation, converting “has this regime been seen?” into the weaker “has a sufficiently similar instance been seen?” and letting the mechanism decline reuse when a regime recurs under visibly different conditions. On reuse the retrieved ensemble becomes the active policy and its soft-voting weights may be recalibrated on the most recent buffered window by measuring the recent accuracy of the two components and renormalizing them; calibration is bounded to the last 50 observations and skipped below 20, so its cost is fixed and small. When no checkpoint exists a new policy is trained from a class-anchored buffer and stored; that buffer retains a stratified slice of the training prefix, applied identically to every adaptive model here, to prevent a refit from collapsing onto one class when the recent window is homogeneous.

C. Monitoring, Refresh, and Fallback

Two monitoring rules are available, and their difference is one finding of this work. The *absolute* rule fires when rolling accuracy falls below a fixed floor; the *relative* rule fires when accuracy has dropped by more than a fixed margin against a decayed reference built from that policy’s earlier accuracy. Section VI-E shows the consequence: a policy that was never good cannot be detected by a relative rule, because the reference decays toward the poor level and the gap the rule looks for never opens. A refresh rebuilds the active policy from the recent buffer and replaces its checkpoint, so later

TABLE I
DATASETS USED IN THE EVALUATION. RECURRENCES COUNT REGIME
RE-VISITS.

Property	Campus	UGR'16	Nordic	5G NR
Samples	1,790	43,200	91,000	499
Features	13	133	15	11
Classes	3	2	3	3
Windows	179	180	182	499
Window size	10	240	500	1 (500pkt)
Regimes	3	12	3	4
Recurrences	14	168	17	6

reuses retrieve the refreshed policy. The default refresh refits the 100-tree ensemble on a 1000-observation buffer; the cost-aware refresh keeps the trigger but refits 20 trees on a 300-observation buffer (Section VI-E).

RAPT-Enhanced adds *buffer-blended novelty refitting*, refitting a novel regime on a 1500-observation buffer rather than 500, and *selective parity refitting*, refitting a reused policy and replacing its checkpoint when rolling accuracy falls below a parity threshold over a short window. Parity refitting addresses the stale-reuse failure directly, since it does not require the regime identifier to change before a retrieved policy can be corrected. When no checkpoint exists and the similarity gate declines an available one, the controller trains a fresh policy and stores it under the incoming regime identifier; this fallback is exactly the full-retraining behaviour RAPT avoids only when unnecessary.

V. EXPERIMENTAL METHODOLOGY

A. Datasets

Four network streams are used, summarised in Table I. **5G Campus Network QoS** is a per-window QoS stream from a 5G campus testbed with 13 features and a three-class target (GOOD, DEGRADED, BAD) from the next window's 90th-percentile delay; regimes are scenario labels and three recur 14 times. **UGR'16** is an ISP netflow archive of per-minute aggregates with 133 features [27]; its calibration split holds only blacklisted traffic with no labelled attacks, so the labelled 30-day block is used, with a binary target and workday/weekend crossed with four-hour time-of-day blocks giving twelve regimes and 168 recurrence events, the strongest recurrence structure here. **NordicDat** is a set of cross-border LTE and 5G predictive-QoS traces [28] with a three-class next-window median-delay target and mobile-operator regimes, the weakest and fewest-transition stream. **5G NR End-to-End Latency** is a simulated dataset [29] of 499 windows of 500 packets spanning four regimes (*ABCDACBDAB*) with the campus QoS target, used for the natural-drift baseline and controlled drift experiments.

B. Preprocessing, Ordering, and Leakage Control

All streams follow one lightweight pipeline: order by timestamp, drop identifier and constant columns, keep numeric features, map infinities to missing, drop rows with missing targets, impute with training-prefix medians, encode categorical targets

deterministically, and window. No PCA, feature selection, hyper-parameter search or global normalization is applied, and target thresholds and scaling statistics are computed on the initial training prefix only. Streams are never shuffled. The prefix is 20% of the windows (36 for the three cross-dataset streams, 100 for 5G NR), is never evaluated as test data, and is the only source of target thresholds. Windows are predicted before their labels are used for adaptation, giving the prequential order **predict** $\rightarrow$ **evaluate** $\rightarrow$ **adapt**; regime annotations are read from the stream, not derived from the target.

C. Models and Detectors

The five primary models share one base architecture and one retraining buffer, so differences reflect only the adaptation mechanism: a heterogeneous soft-voting ensemble of a random forest and an extra trees classifier, each 50 trees of depth 7. *Frozen* trains once on the prefix and never adapts; *Event-Driven* retrains when window accuracy drops below a fixed threshold; *Full Retraining* rebuilds at every regime transition; *RAPT* and *RAPT-Enhanced* are the controllers of Section IV.

ADWIN, Page-Hinkley, EDDM and ECDD-EWMA drive the same retraining harness with identical model, buffer, capacity and minimum retrain interval, so only the trigger differs. Parameters are fixed a priori and not tuned on the test stream (ADWIN $\delta = 0.002$; Page-Hinkley minimum 30 instances, $\delta = 0.005$; EDDM $\alpha = 0.95$, $\beta = 0.9$; ECDD-EWMA $\lambda = 0.2$, $L = 2$). As ground-truth boundaries are unavailable, we report detected events, adaptation cost and predictive performance rather than detector precision and recall.

D. Metrics and Statistics

Predictive performance is reported as macro-F1, accuracy, precision and recall. Adaptation cost is reported as *adaptation CPU*, the time inside adaptation routines only, and *runtime*, the total streaming time; these are kept separate because a reduction in adaptation time does not translate proportionally into total runtime. We also record retraining and reuse events and trees trained or reused. All experiments use seeds [42, 43, 44, 45, 46] and report mean $\pm$ standard deviation; paired comparisons use the Wilcoxon signed-rank test with Cohen's d . Window pairing has 143–146 observations, seed pairing $n = 5$ and a smallest attainable two-sided p -value of 0.0625; that ceiling is stated wherever it binds, and non-significant results are reported as a failure to establish a difference rather than as equivalence. Measurements used a single process with `n_jobs=1`, so timings are comparable within the study but not portable. All four streams are evaluated by one harness under the configuration of Table I.

VI. RESULTS

A. Predictive Performance

Table II reports the primary comparison and Fig. 1 shows the macro-F1 values side by side.

TABLE II

PRIMARY COMPARISON AND COST, MEAN OVER SEEDS 42–46. BOLD MARKS THE BEST MACRO-F1 PER STREAM; CPU AND RUNTIME IN SECONDS, EVENTS ARE COUNTS.

Dataset	Model	F1	Acc.	Prec.	Rec.	CPU (s)	Run (s)	Retr.	Reuse
5G Campus	Frozen	0.9370 $\pm$ 0.0010	0.9822 $\pm$ 0.0006	0.9386 $\pm$ 0.0010	0.9383 $\pm$ 0.0003	0.0000	1.2397	0	–
	Event-Driven	0.9610 $\pm$ 0.0030	0.9898 $\pm$ 0.0018	0.9609 $\pm$ 0.0033	0.9627 $\pm$ 0.0019	0.1171	1.3534	1	–
	Full Retraining	0.9739 $\pm$ 0.0049	0.9929 $\pm$ 0.0013	0.9741 $\pm$ 0.0044	0.9751 $\pm$ 0.0049	1.2779	2.5167	13	–
	RAPT	0.9402 $\pm$ 0.0028	0.9831 $\pm$ 0.0017	0.9421 $\pm$ 0.0012	0.9414 $\pm$ 0.0032	0.2568	1.4766	2	11
	RAPT-Enhanced	0.9402 $\pm$ 0.0028	0.9831 $\pm$ 0.0017	0.9421 $\pm$ 0.0012	0.9414 $\pm$ 0.0032	0.2628	1.5065	2	11
UGR'16	Frozen	0.9688 $\pm$ 0.0038	0.9905 $\pm$ 0.0006	0.9784 $\pm$ 0.0034	0.9661 $\pm$ 0.0037	0.0000	1.3203	0	–
	Event-Driven	0.9104 $\pm$ 0.0211	0.9899 $\pm$ 0.0019	0.9177 $\pm$ 0.0205	0.9081 $\pm$ 0.0217	0.6537	2.0142	3	–
	Full Retraining	0.9591 $\pm$ 0.0031	0.9877 $\pm$ 0.0010	0.9722 $\pm$ 0.0022	0.9576 $\pm$ 0.0027	23.6137	25.0668	143	–
	RAPT	0.8503 $\pm$ 0.0346	0.9565 $\pm$ 0.0208	0.8688 $\pm$ 0.0321	0.8491 $\pm$ 0.0354	2.7376	4.0972	11	132
	RAPT-Enhanced	0.8988 $\pm$ 0.0412	0.9781 $\pm$ 0.0205	0.9131 $\pm$ 0.0375	0.8972 $\pm$ 0.0402	3.2996	4.6586	11	132
NordicDat	Frozen	0.3037 $\pm$ 0.0182	0.4843 $\pm$ 0.0074	0.4479 $\pm$ 0.0183	0.2767 $\pm$ 0.0191	0.0000	1.4280	0	–
	Event-Driven	0.2735 $\pm$ 0.0309	0.4620 $\pm$ 0.0449	0.4059 $\pm$ 0.0186	0.2476 $\pm$ 0.0273	0.2282	1.6855	2	–
	Full Retraining	0.4765 $\pm$ 0.0171	0.5920 $\pm$ 0.0077	0.5498 $\pm$ 0.0159	0.4629 $\pm$ 0.0180	2.1731	3.6260	18	–
	RAPT	0.3978 $\pm$ 0.0361	0.6037 $\pm$ 0.0099	0.4863 $\pm$ 0.0328	0.3767 $\pm$ 0.0372	0.3146	1.7711	2	16
	RAPT-Enhanced	0.4644 $\pm$ 0.0332	0.5996 $\pm$ 0.0120	0.5411 $\pm$ 0.0272	0.4494 $\pm$ 0.0343	1.4094	2.8665	2	10
5G NR Lat.	Frozen	0.9048 $\pm$ 0.0056	0.9048 $\pm$ 0.0056	0.9048 $\pm$ 0.0056	0.9048 $\pm$ 0.0056	0.0000	3.4044	0	–
	Event-Driven	0.8576 $\pm$ 0.0045	0.8576 $\pm$ 0.0045	0.8576 $\pm$ 0.0045	0.8576 $\pm$ 0.0045	1.1178	4.5229	13	–
	Full Retraining	0.8586 $\pm$ 0.0074	0.8586 $\pm$ 0.0074	0.8586 $\pm$ 0.0074	0.8586 $\pm$ 0.0074	0.6070	4.0174	7	–
	RAPT	0.7945 $\pm$ 0.0163	0.7945 $\pm$ 0.0163	0.7945 $\pm$ 0.0163	0.7945 $\pm$ 0.0163	0.2901	3.7132	3	4
	RAPT-Enhanced	0.8446 $\pm$ 0.0106	0.8446 $\pm$ 0.0106	0.8446 $\pm$ 0.0106	0.8446 $\pm$ 0.0106	0.5998	4.0036	3	4

On 5G Campus all five models are close to saturation: Full Retraining recorded the highest macro-F1 (0.9739 ± 0.0049), and RAPT and RAPT-Enhanced both recorded 0.9402 ± 0.0028 , 0.0338 below it and significant in that direction ($p = 0.0625$, $d_z = -13.05$); the two variants agree to four decimals, so the enhancement had no measurable effect here. UGR'16 is where the mechanism's assumptions break down and is the most informative result: Frozen recorded the highest macro-F1 (0.9688 ± 0.0038), consistent with attacks being sparse and clustered, while RAPT recorded 0.8503 ± 0.0346 , 0.1088 below Full Retraining ($p = 0.0625$, $d_z = -3.16$) and the lowest of the five models despite 132 reuses. The twelve regimes recur and their identifiers stay valid across the 30-day block, but the relationship between netflow features and attack labels does not, so reuse retrieves policies matching the regime label but not its label semantics. RAPT-Enhanced recovered to 0.8988 ± 0.0412 , a gain of 0.0485 over RAPT ($p = 0.0625$, $d_z = 1.05$). An ablation shows this gain is attributable to the larger novelty-refit buffer ($500 \rightarrow 1500$ observations), not to the parity trigger: at its default threshold the parity refit fires zero times, and raising the base novelty buffer to 1500 alone reproduces 0.8988.

NordicDat is the weakly predictable stream: Frozen recorded 0.3037 ± 0.0182 while adaptation helps, with Full Retraining at 0.4765 ± 0.0171 and RAPT at 0.3978 ± 0.0361 , a deficit of 0.0786 that is significant at the seed level ($p = 0.0625$, $d_z = -4.04$); RAPT did record the highest accuracy here (0.6037 ± 0.0099 against 0.5920 ± 0.0077), a precision-favouring decision profile rather than better macro-F1. On 5G NR the five models span 0.1103, with Frozen highest (0.9048 ± 0.0056) and RAPT 0.0642 below Full Retraining. The predictive results do not support a claim that RAPT improves accuracy: Full Retraining records the highest macro-F1 on two of four streams and RAPT is below it on all four,

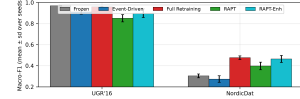


Fig. 1. Macro-F1 on the two largest cross-dataset streams. RAPT tracks the frozen baseline on UGR'16 rather than the retrained models, the signature of stale policy reuse; on the 5G NR stream the five models span 0.1103.

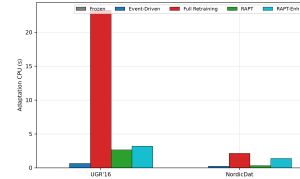


Fig. 2. Adaptation CPU on the two largest streams. The ordering is inverted relative to Fig. 1: RAPT uses an order of magnitude less adaptation time than Full Retraining on UGR'16 while recording lower macro-F1.

so the case rests on cost and on when reuse is safe.

B. Computational Cost and Policy Reuse

Table II also reports adaptation and total cost, and Fig. 2 shows the adaptation-CPU comparison. The reduction relative to Full Retraining is consistent across all four streams: 1.2779 to 0.2568 s on 5G Campus (−79.9%), 23.6137 to 2.7376 s on UGR'16 (−88.4%), 2.1731 to 0.3146 s on NordicDat (−85.5%), and 0.6070 to 0.2901 s on 5G NR (−52.2%). Runtime reductions are smaller in every case (−42.3%, −83.6%, −51.1%, −1.5%), the gap being prediction and feature-construction cost. Full Retraining retrains at every regime transition (13, 143, 18 and 7), while RAPT performs 2, 11, 2 and 3 retrains against 11, 132, 16 and 4 retrievals.

TABLE III

RAPT EFFICIENCY ABLATION ON THE 5G CAMPUS STREAM, MEAN OVER SEEDS 42–46. REFRESHES COUNTS BOUNDED IN-PLACE POLICY REBUILDS; RAPT-INCREMENTAL INSTEAD UPDATES AN ONLINE LEARNER.

Configuration	Macro-F1	Adapt. CPU	Refreshes
Full Retrain. (ref.)	0.9739 ± 0.0049	1.2779	–
RAPT-Full	0.9537 ± 0.0064	0.6211	0
RAPT-Evidence	0.9402 ± 0.0028	0.2582	0
RAPT-Floor	0.9402 ± 0.0028	0.2568	0
RAPT-Periodic-FullRefit	0.9745 ± 0.0044	4.6486	22
RAPT-Cheap	0.9717 ± 0.0081	0.9801	22
RAPT-Incremental	0.7164 ± 0.0057	0.1874	130

C. Cross-Dataset Behaviour

The adaptation-cost reduction and the predominance of reuse appear on all four streams, so both are properties of the mechanism; the predictive outcome is not. RAPT-Enhanced improves over base RAPT by 0.0485 on UGR’16, 0.0666 on NordicDat and 0.0501 on 5G NR and is identical on 5G Campus, so the enhancement matters only where regimes recur and their label relationship changes. Reuse frequency alone says nothing about whether a retrieved policy was appropriate: on UGR’16 both RAPT and RAPT-Enhanced reuse 132 times, yet RAPT records the lowest macro-F1 of the five models and RAPT-Enhanced 0.8988, locating the failure in the reuse decision rather than in reuse as such. The NordicDat orderings of accuracy and macro-F1 invert (0.6037 against 0.5920), which cannot be resolved with the present data.

D. Efficiency Ablation

Table III reports the mechanism ladder on 5G Campus. RAPT-Full recorded 0.9537 ± 0.0064 at 0.6211 s; replacing its trigger with a relative-degradation one (RAPT-Evidence) fired *zero* refreshes and left macro-F1 at 0.9402 ± 0.0028 ; an absolute floor (RAPT-Floor) also fired zero refreshes at 0.9402 ± 0.0028 , because Campus window accuracy never falls below the 0.60 floor. The periodic schedule is what actually refreshes: with a full refit every five windows it recorded 0.9745 ± 0.0044 at 4.6486 s (22 refreshes), and with the cheaper refit RAPT-Cheap recorded 0.9717 ± 0.0081 at 0.9801 s against Full Retraining’s 0.9739 ± 0.0049 at 1.2779 s. The difference of -0.0022 is not significant ($p = 0.4375$, $d_z = -0.34$), so it does not establish a difference; relative to the full-refit periodic refresh it keeps accuracy while cutting adaptation CPU by roughly 79%. RAPT-Incremental (0.7164 at 0.1874 s) is dominated on both axes. The supported statement is therefore *matched* accuracy at substantially lower cost, not superiority.

E. Cost Drivers and Trigger Behaviour

The cheap refresh was chosen from direct measurement. Varying one factor at a time on the base ensemble, a 20-tree refit on a 1000-observation buffer cost 65.8 ms against 320.1 ms for 100 trees on the same buffer, while holding tree count at 20 and growing the buffer from 200 to 1000 changed the cost from 37.3 to 65.8 ms; tree count therefore



Fig. 3. Historical detector comparison on the published signal and wiring. ADWIN, Page-Hinkley and ECDD-EWMA never fire under this configuration and reproduce the frozen baseline, while EDDM is the only detector that adapts on more than one stream; RAPT adapts least among methods that adapt. The near-zero counts are a property of the signal and the throttle, not of the detectors.

dominated buffer size. Table III places the RAPT variants on the accuracy–cost plane for this stream.

The zero-refresh result for RAPT-Evidence (Section VI-D) is mechanism-level rather than a tuning outcome. A relative trigger compares current accuracy against a decayed reference built from that policy’s own past. When a regime is persistently poor rather than deteriorating, the reference decays toward the poor level and the gap the rule tests for never opens, so the trigger stays silent even though the policy is unfit. On 5G Campus neither the relative rule nor the absolute floor fires, because window accuracy never drops below the floor; on NordicDat and 5G NR, where accuracy does dip, both rules fire, with the relative rule firing more often (31.4 and 43.4 refreshes against 28.2 and 15.6). The floor threshold is stream-specific and would require calibration elsewhere.

F. Historical Detector Comparison

Under the shared harness the reported counts are as follows. On the window error signal (143–146 windows for the cross-dataset streams) every detector fires at most twice: ADWIN, Page-Hinkley and ECDD-EWMA fire zero times on all four streams and EDDM fires 0/0/2/1 times (Campus/UGR’16/NordicDat/5G NR), which the retrain throttle turns into 0, 6.8, 6.2 and 1.0 retrains. These near-zero counts are largely a signal-scale artefact rather than detector inertness: fed their native per-sample error bits the same detectors fire far more often, with ADWIN at 0/12/164/0, Page-Hinkley at 0/2/103/0 and EDDM at 0/8/111/1. Detector numbers therefore reflect what the window aggregate exposes, and EDDM is the only detector that adapts on more than one stream. RAPT recorded 2, 11, 2 and 3 adaptation events at 0.2568, 2.7376, 0.3146 and 0.2901 s, cheaper than the full retrain on all four streams.

G. Statistical Analysis

The paired comparisons support a narrow set of conclusions. Under seed-level pairing ($n = 5$) RAPT’s macro-F1 deficit against Full Retraining is significant at the floor $p = 0.0625$ on all four streams, with paired effect sizes d_z of -13.05 (Campus), -3.16 (UGR’16), -4.04 (NordicDat) and -4.58 (5G NR); the moving-block bootstrap over windows agrees on all four. RAPT-Enhanced’s recovery over RAPT is likewise significant at $p = 0.0625$ on UGR’16 ($+0.0485$, $d_z = 1.05$), NordicDat ($+0.0666$, $d_z = 4.00$) and 5G NR ($+0.0501$, $d_z = 5.44$) and is exactly zero on 5G Campus. The five-seed ablation comparisons are limited by power as well as effect

size: RAPT-Cheap’s difference from Full Retraining (-0.0022 , $p = 0.4375$, $d_z = -0.34$) and from the full-refit periodic refresh (-0.0027 , $p = 0.4375$) are both non-significant, so they establish no difference rather than equivalence. One caveat applies throughout: window-level tests treat windows from one stream as independent, overstating the effective sample size.

Because five seeds cap the attainable two-sided p -value at 0.0625, we repeated the cheap configurations at twenty seeds (42–61) with a Holm correction. All eight Holm-corrected comparisons are significant: RAPT-Cheap beats the cheap periodic refit and the cheap full retrain on 5G Campus ($+0.0024$, $+0.0021$) and UGR’16 ($+0.1572$, $+0.0921$), and loses on 5G NR (-0.0195 , -0.0116) and NordicDat (-0.0401 , -0.0588), all with Holm-adjusted $p < 0.01$.

H. Label-Delay Sensitivity

Latency telemetry reveals a window’s target only one window later. With that delay, 5G Campus is unchanged and 5G NR changes negligibly ($|\Delta| \leq 0.0064$), but NordicDat degrades for every adapting method, most for RAPT ($0.4618 \rightarrow 0.3362$, $p = 0.0625$) and Full Retraining ($0.4806 \rightarrow 0.4205$); RAPT’s NordicDat advantage does not survive the delay.

VII. DISCUSSION

The cost result has a straightforward explanation. Retraining at a regime transition rebuilds the ensemble from a buffer, at a cost scaling with tree count and buffer size, whereas reuse retrieves an ensemble stored by reference for a dictionary lookup plus a bounded weight calibration. Where regimes recur with stable label relationships the rebuild is unnecessary, so the saving tracks how often regimes recur. RAPT-Cheap works because tree count dominates refit cost, so a cheap refresh corrects a nearly adequate policy, and the periodic schedule is what makes it effective, since a reused policy would otherwise stay frozen for the whole regime.

The two rejected mechanisms failed for identifiable reasons. The incremental learner recorded 0.7164 macro-F1 at 0.1874 s, dominated on both axes by the batch variants, and the batch ensemble is already near ceiling on a 13-feature, three-class problem, so this says more about this stream than about incremental learning generally. The trigger finding generalises more cleanly: a relative trigger compares a policy against its own past, so a policy that was always poor has no relative drop to detect, a structural blind spot rather than a tuning problem.

Recurrence is the condition under which the mechanism is worth deploying. The cost saving appears wherever reuse occurs, but the accuracy penalty is concentrated on UGR’16, where regimes recur while their label relationship does not: precisely where a reused policy is stale. The repair we measured there came from a larger novelty-refit buffer (500 $\rightarrow$ 1500 observations), which lifted RAPT from 0.8503 to 0.8988, rather than from the parity trigger, which fires zero times at its default threshold. Reuse on the regime identifier alone is therefore unsafe when label semantics can drift independently

of it, and a stream without recurrence reduces to retraining with extra bookkeeping.

VIII. LIMITATIONS AND THREATS TO VALIDITY

Five seeds cap the attainable two-sided p -value at 0.0625, so several large-effect comparisons remain formally non-significant and the ablation results are limited by power; the twenty-seed comparisons are the exception. Window-level tests treat windows from one stream as independent, overstating the effective sample size, and timings are comparable within the study but not portable.

Three measurement caveats bound the quantitative claims. The detector ordering reflects a window-level error signal and a retrain throttle; on a per-sample signal ADWIN and Page-Hinkley do fire, so the comparison characterises a configuration rather than the detectors. The 5G NR row is a 500-packet window whereas the Campus stream is 1,790 samples over 179 ten-packet windows, so per-window resolution differs across streams. The reported gain of RAPT-Enhanced over RAPT on UGR’16 is confounded with the novelty-refit buffer size, which the parity trigger does not control.

Further limitations follow from the design. The mechanism depends on regime recurrence, which is measured rather than assumed; a stream without recurrence never exercises the repository. The RAPT-Floor threshold is calibrated on the evaluated stream and the 20-tree cheap refit is chosen for this ensemble, so both would need re-examination where the pre-/post-drift difference is larger. The repository is never pruned, and the approach assumes regime similarity is meaningful, which could be questionable if the regime signal were noisy.

The controlled drift-severity study (severity definitions, evidence that the concept-drift transformation changes $P(Y | X)$ rather than $P(X)$, and the resulting curves) is reported in the supplementary material. Its concept-drift isolation uses a boundary-only controller with no in-regime trigger, so the inertness it records under regime-internal concept drift bounds that controller rather than the monitored variant described here.

IX. CONCLUSION

Retraining is the conventional drift remedy, and where conditions recur that expenditure is partly redundant. RAPT retains each regime’s policy and retrieves it when the regime returns, retraining only when no suitable policy exists and refreshing one that monitoring shows has stopped fitting. Across four network streams it reduced adaptation CPU by 53–88% versus full retraining but recorded lower macro-F1 on all four, and where regimes recur with changed label relationships blind reuse was harmful, recovering only with a larger novelty-refit buffer. Reuse is therefore worth deploying where regimes recur with stable or monitored label semantics; the natural extension is a reuse decision that tests whether a retrieved regime is still semantically the same.

REFERENCES

- [1] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey on concept drift adaptation,” *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014.

- [2] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Trans. Knowledge and Data Engineering*, vol. 31, no. 12, pp. 2346–2363, 2018.
- [3] I. Khamassi, M. Sayed-Mouchaweh, M. Hammami, and K. Ghédira, "Discussion and review on evolving data streams and concept drift adapting," *Evolving Systems*, vol. 9, no. 1, pp. 1–23, 2018.
- [4] J. C. Schlimmer and R. H. Granger, Jr., "Incremental learning from noisy data," *Machine Learning*, vol. 1, no. 3, pp. 317–354, 1986.
- [5] G. Widmer, "Combining robustness and flexibility in learning drifting concepts," in *Proc. European Conf. Artificial Intelligence (ECAI)*, 1994, pp. 468–472.
- [6] H. M. Gomes, A. Bifet, J. Read, J. P. Barddal, F. Enembreck, B. Pharinger, G. Holmes, and T. Abdessalem, "Adaptive random forests for evolving data stream classification," *Machine Learning*, vol. 106, no. 9–10, pp. 1469–1495, 2017.
- [7] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proc. SIAM Int. Conf. Data Mining (SDM)*, 2007, pp. 443–448.
- [8] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.
- [9] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Proc. Brazilian Symp. Artificial Intelligence (SBIA)*, 2004, pp. 286–295.
- [10] M. Baena-García, J. del Campo-Ávila, R. Fidalgo, A. Bifet, R. Gavaldà, and R. Morales-Bueno, "Early drift detection method," in *Proc. Int. Workshop on Knowledge Discovery from Data Streams*, 2006, pp. 77–86.
- [11] R. S. M. Barros, D. R. L. Cabral, P. M. Gonçalves, Jr., and S. G. T. C. Santos, "RDDM: Reactive drift detection method," *Expert Systems with Applications*, vol. 90, pp. 344–355, 2017.
- [12] G. J. Ross, N. M. Adams, D. K. Tasoulis, and D. J. Hand, "Exponentially weighted moving average charts for detecting concept drift," *Pattern Recognition Letters*, vol. 33, no. 2, pp. 191–198, 2012.
- [13] I. Frias-Blanco, J. del Campo-Ávila, G. Ramos-Jimenez, R. Morales-Bueno, A. Ortiz-Diaz, and Y. Caballero-Mota, "Online and non-parametric drift detection methods based on Hoeffding's bounds," *IEEE Trans. Knowledge and Data Engineering*, vol. 27, no. 3, pp. 810–823, 2015.
- [14] D. M. dos Reis, P. Flach, S. Matwin, and G. Batista, "Fast unsupervised online drift detection using incremental Kolmogorov–Smirnov test," in *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 1545–1554.
- [15] T. Brockhoff, M. S. Uysal, and W. M. P. van der Aalst, "Time-aware concept drift detection using the earth mover's distance," in *Proc. Int. Conf. Process Mining (ICPM)*, 2020, pp. 33–40.
- [16] L. Yang and A. Shami, "A lightweight concept drift detection and adaptation framework for IoT data streams," *IEEE Internet of Things Magazine*, vol. 4, no. 2, pp. 96–101, 2021.
- [17] J. Z. Kolter and M. A. Maloof, "Dynamic weighted majority: An ensemble method for drifting concepts," *Journal of Machine Learning Research*, vol. 8, no. 91, pp. 2755–2790, 2007.
- [18] D. P. Helmbold and P. M. Long, "Tracking drifting concepts by minimizing disagreements," *Machine Learning*, vol. 14, no. 1, pp. 27–45, 1994.
- [19] D. Brzeziński and J. Stefanowski, "Reacting to different types of concept drift: The accuracy updated ensemble algorithm," *IEEE Trans. Neural Networks and Learning Systems*, vol. 25, no. 1, pp. 81–94, 2014.
- [20] A. Abbasi, A. R. Javed, C. Chakraborty, J. Nebhen, W. Zehra, and Z. Jalil, "ElStream: An ensemble learning approach for concept drift detection in dynamic social big data stream learning," *IEEE Access*, vol. 9, pp. 66408–66419, 2021.
- [21] L. Yang, D. M. Manias, and A. Shami, "PWPAA: An ensemble framework for concept drift adaptation in IoT data streams," in *Proc. IEEE Global Communications Conf. (GLOBECOM)*, 2021, pp. 1–6.
- [22] P. Domingos and G. Hulten, "Mining high-speed data streams," in *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2000, pp. 71–80.
- [23] J. Montiel, M. Halford, S. M. Mastelini, G. Bolmier, R. Sourty, R. Vaysse, A. Zouitine, H. M. Gomes, J. Read, T. Abdessalem, and A. Bifet, "River: machine learning for streaming data in Python," *Journal of Machine Learning Research*, vol. 22, no. 110, pp. 1–8, 2021.
- [24] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Trans. Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2010.
- [25] S. Markovitch and P. D. Scott, "The role of forgetting in learning," in *Machine Learning Proceedings 1988*. San Francisco, CA, USA: Morgan Kaufmann, 1988, pp. 459–465.
- [26] M. Tambe and A. Newell, "Some chunks are expensive," in *Machine Learning Proceedings 1988*. San Francisco, CA, USA: Morgan Kaufmann, 1988, pp. 451–458.
- [27] G. Maciá-Fernández, J. Camacho, R. Magán-Carrión, P. García-Teodoro, and R. Therón, "UGR'16: A new dataset for the evaluation of cyclostationarity-based network IDSs," *Computers & Security*, vol. 73, pp. 411–424, 2018.
- [28] NordicDat cross-border predictive QoS dataset, Zenodo record 10964584, released under CC-BY-4.0.
- [29] 5G NR end-to-end latency simulation dataset, Zenodo record 20035549.